Retrieval Engine Competition Project

Final Report for Submission

Group: SeaFour

Maksym DOLHOV
Mehdi AGHAEI
Nguyen Ho Bao KHANH

Submission scope. This final version is written to match the completed notebook `kaggle-submission.ipynb`. It reports the final dataset exploration, preprocessing choices, the corrected query-split protocol, Phase 1 retrieval comparison, Phase 2 classifier-aware reranking results, document-holdout classification, and the regenerated Kaggle submission file.

Main notebook	`kaggle-submission.ipynb`
Main output	`solutions_SeaFour.csv`
Dataset source	Kaggle competition files provided for the course
Latest confirmed public Kaggle score	**0.60007** (corrected notebook submission)
Date	March 29, 2026

1 Introduction

Information retrieval is the task of ranking documents so that the most relevant ones appear first for a user query. In this project we work with a large technical corpus and a set of labeled training queries. The assignment is divided into two phases. Phase 1 focuses on basic retrieval with classical and neural methods. Phase 2 adds category classification and asks how the classifier can be used to improve retrieval.

The final notebook is structured around the required project questions rather than around only a single optimized run. It therefore contains:

- dataset loading and statistical exploration;
- construction of a normalized `content` field;
- three retrieval models: TF-IDF, BM25+, and Sentence-Transformer embeddings;
- embedding inspection through matrix shapes and a 2D projection;
- evaluation with Recall@K, Precision@K, and MRR@K;
- category classification for both documents and queries;
- classifier-aware reranking for Phase 2;
- Kaggle submission export.

2 Project Roadmap

This report is meant to document not only the final score, but also the path that led to it. The project started with basic lexical retrieval, then moved toward semantic retrieval, then added category prediction as a second signal, and finally became a submission-ready pipeline after several debugging and reproducibility fixes.

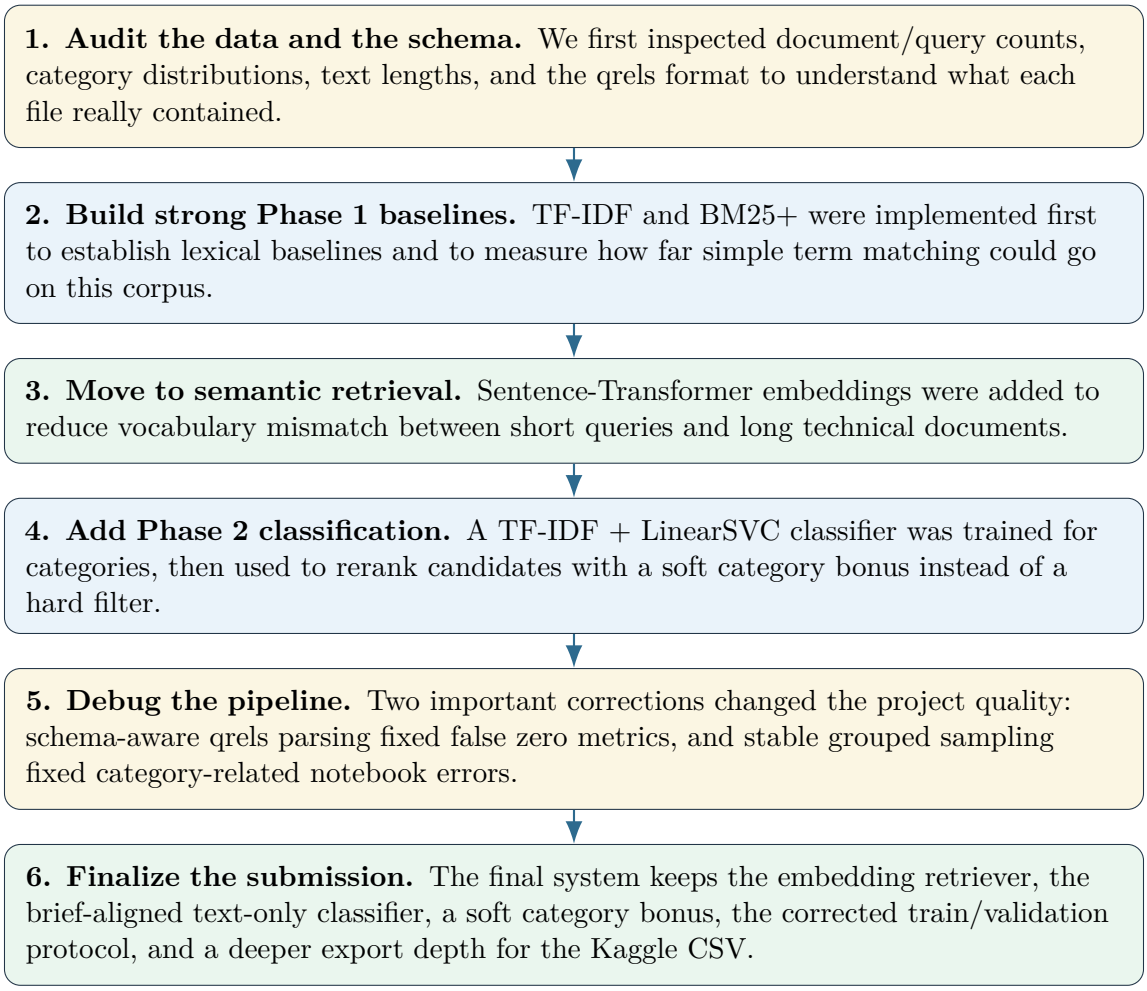

Figure 1: Roadmap from the first baselines to the final submission-ready system.

Stage	Main question	Outcome
Data audit	What is in the files, and how should they be interpreted?	We clarified the schema, the category structure, the query/document asymmetry, and the relevance format used by the qrels.
Lexical baselines	How far can exact-term retrieval go?	TF-IDF and BM25+ gave interpretable baselines, but they remained limited by lexical mismatch.
Dense retrieval	Can semantic matching improve recall and rank quality?	Embeddings became the strongest retriever by a large margin on Recall and MRR.
Classification	Can category prediction help retrieval?	Yes, but as a soft reranking signal rather than as a strict filter.
Debugging	Why did some notebook outputs look suspicious or fail?	Fixing qrels parsing and grouped sampling removed false zeros and notebook instability.
Submission	Which pipeline is strong and defensible enough to submit?	Embeddings + text-only classifier + soft bonus was kept as the final system.

Table 1: Milestones that structured the project from start to final submission.

3 Final Changes and Reasons

The final submission differs from earlier draft states of the project in five important ways.

1. **Ground-truth parsing was aligned with the real qrels schema.** The competition qrels store

relevant items under `relevant_doc_ids` with nested `doc_id` entries. Aligning the loader with that format was necessary so that Recall, Precision, and MRR reflect the true labels instead of empty relevance sets. The final notebook now also raises an explicit error if qrels parsing yields empty relevance, missing labeled-query coverage, or document ids that are absent from the corpus.

2. **Category-wise sampling was made stable for current pandas behavior.** The notebook now uses a sampling pattern that preserves the `category` column reliably during visualization and downsampling steps. This prevents category-aware plots and classifier preparation from failing because of grouped-sampling side effects.

3. **The query split protocol was corrected.** Only `queries_train.json` comes with visible qrels through `qgts_train.json`, while `queries_test.json` is the unlabeled competition test set. For that reason, presenting an internal `60 / 30 / 10` query split as train / validation / test was misleading. The corrected notebook now uses a train / validation split on labeled queries and reserves `queries_test.json` for final export only.

4. **The report now uses measured values instead of placeholders.** All result tables below are populated from the completed notebook outputs, with query-side evaluation on validation and document-side holdout classification kept separate.

5. **The final active system was chosen for clarity and empirical strength.** The selected submission pipeline is the embedding retriever with the brief-aligned `text_only` classifier and a soft category bonus. This choice is easy to justify academically and achieved the best Phase 2 validation score among the tested active configurations.

3.1 Previous tries, failures, and lessons

Type	Attempt or issue	What happened	Decision or lesson
Attempt	TF-IDF baseline	The model was simple and useful as a baseline, but lexical overlap alone was not enough for many relevant pairs.	Keep it as a baseline, not as the final retriever.
Attempt	BM25+ lexical upgrade	BM25+ improved over TF-IDF and handled length variation better, but it still lagged far behind embeddings on recall and rank quality.	Use it as the strongest lexical reference point.
Failure	Qrels parsing mismatch	The notebook read the wrong relevance field, which produced empty relevance sets and misleading zero Recall, Precision, and MRR values.	Parse `relevant_doc_ids` correctly, verify average relevant documents per query, and fail early if the parsed qrels no longer match the labeled queries or corpus ids.
Failure	Grouped sampling instability	A grouped pandas sampling pattern sometimes dropped the `category` column and caused notebook errors such as `KeyError: category`.	Switch to stable category-preserving downsampling for visualization and classifier preparation.
Attempt	Query classifier with `text + tags`	The classifier became extremely accurate, but this setting uses metadata that we did not want to silently turn into the default Phase 2 pipeline.	Keep it as an explicit ablation and use text-only as the active setting.
Decision	Validation depth vs. submission depth	The best offline evaluation depth was $K = 1000$, but the final CSV benefits from exporting a deeper ranked list.	Separate validation selection from submission export and write 7,500 ids per query in the final file.

Table 2: Main previous tries, failures, and lessons learned during the project.

4 Dataset Description and Exploratory Analysis

The dataset contains documents, training queries, test queries, and relevance judgments. From the final project files, the corpus contains **216,041 documents**, **327 labeled training queries**, and **141 Kaggle test queries**. The documents belong to **five categories**. Because only the training queries have visible qrels, the corrected notebook uses those 327 labeled queries for a train / validation split and keeps the Kaggle test file untouched for submission. Tables 3–5 summarize the global scale of the collection, the corrected split sizes used in the notebook, and the category distribution.

Dataset element	Count or value
Documents	216,041
Labeled training queries	327
Kaggle test queries	141
Categories	5
Average relevant documents per training query	9.08
Average document length	900.92 characters / 94.86 tokens
Average training query length	48.44 characters / 4.95 tokens
Average test query length	48.88 characters / 4.96 tokens

Table 3: Global dataset overview.

Split	Number of labeled queries
Split-train	228
Validation	99

Table 4: Query split sizes used by the corrected final notebook.

Category	Document count	Training-query count
`tex`	68,184	130
`unix`	47,382	47
`gaming`	45,301	41
`programmers`	32,176	52
`android`	22,998	57

Table 5: Document and labeled-query distribution by category.

These exploratory results matter because they help explain later retrieval behavior. In particular, the corpus contains long documents while the queries are much shorter. This asymmetry generally makes strict lexical overlap less reliable, especially when the query paraphrases the document instead of reusing its exact surface vocabulary.

4.1 Available fields

Each document contains an `id` and can include `title`, `text`, `tags`, and `category`. Training queries also contain `id`, `title`, `text`, `tags`, and `category`. Test queries omit the category label because that is one of the targets of Phase 2 inference. The qrels file maps each labeled training query to a set of relevant document ids, and the final loader validates that this mapping is non-empty, aligned with `queries_train.json`, and consistent with the document corpus.

5 Preprocessing and Content Construction

The assignment asks for a new text field called `content`. The final notebook uses the following policy:

- documents: `title + text + tags`;
- retrieval queries: `title + text`;
- primary classifier queries: `title + text`;
- classifier ablation: `title + text + tags`.

This design keeps both retrieval and the default Phase 2 classifier tied to the actual query text, which is the safer reading of the brief. Because the raw query files also contain `tags`, the notebook reports a second text-plus-tags classifier as an explicit ablation. That comparison makes the effect of tags visible instead of silently baking it into the main pipeline.

The shared normalization steps are:

- lowercasing;
- replacement of the separators `-`, `_`, and `/` with spaces;
- whitespace collapsing;
- trimming leading and trailing whitespace.

For lexical methods, tokenization uses the pattern `[a-z0-9]+`, and English stopword filtering is enabled by default.

6 Phase 1: Retrieval Models

6.1 TF-IDF retrieval

TF-IDF stands for *term frequency–inverse document frequency*. It gives high weight to terms that appear frequently in one document but are relatively rare in the collection. In vector-space retrieval, each document and query is represented by a sparse TF-IDF vector, and documents are ranked by cosine similarity with the query vector.

Conceptually, TF-IDF is useful because it is simple, fast, and often surprisingly competitive as a baseline. Its main limitation is that it depends strongly on lexical overlap. If a relevant document uses different wording from the query, TF-IDF may fail even when the underlying meaning is similar.

6.2 BM25+ retrieval

BM25+ is a probabilistic lexical ranking model. It improves over naive bag-of-words weighting by:

- saturating term frequency contributions;
- normalizing for document length;
- adding a positive lower-bound adjustment through the BM25+ variant.

BM25+ is often stronger than plain TF-IDF when document lengths vary substantially, as is the case in this corpus. However, BM25+ remains a lexical model and therefore still depends on token overlap.

6.3 Embedding retrieval

Dense embeddings map text into high-dimensional vectors that encode semantic information. In this project we use the Sentence-Transformer model `all-MiniLM-L6-v2`. Documents and queries are embedded into a 384-dimensional space, and cosine similarity is computed through normalized dot products.

Embeddings can help retrieval because they are more robust to synonyms, paraphrases, and broader semantic similarity. Their main tradeoffs are higher computational cost, larger memory usage, and the risk of retrieving semantically related but not strictly relevant documents.

7 Embedding Inspection and Visualization

After generating embeddings, the notebook prints the matrix shapes for documents and queries. This directly answers the specification that the embedding structures should be examined.

To make the geometry more interpretable, the notebook also builds a 2D projection using **UMAP** when available and otherwise falls back to **t-SNE**. The plot uses a category-aware sample of documents together with the training queries. In typical runs, one should observe partial clustering by category. This is expected because categories such as `android`, `gaming`, `tex`, `unix`, and `programmers` tend to contain domain-specific vocabulary and recurrent semantic themes.

At the same time, perfect separation should not be expected. The original embeddings live in a much higher-dimensional space, some categories overlap semantically, and short queries are often more ambiguous than documents.

Figure 2 shows the 2D embedding projection produced by the final notebook. The visible grouping confirms that the dense representation captures meaningful category structure, while the overlaps also show that the classes are not trivially separable in two dimensions.

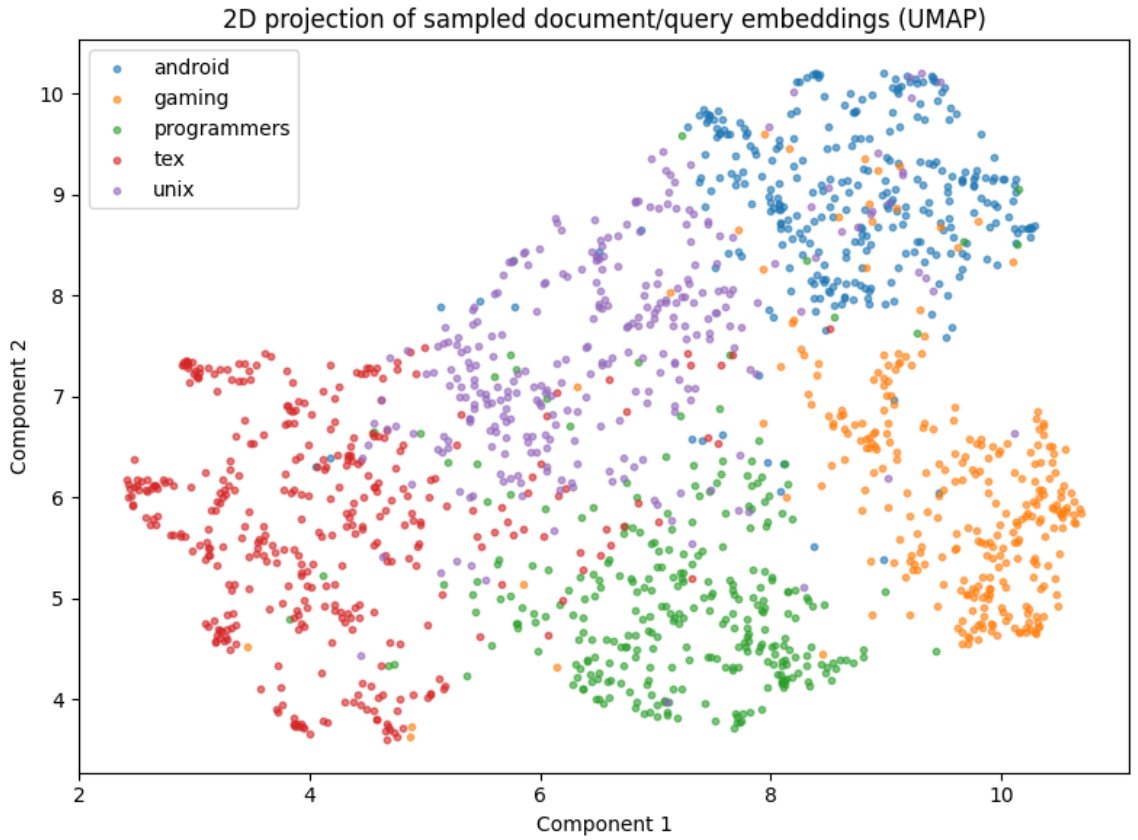

Figure 2: 2D projection of sampled document and training-query embeddings produced by the final notebook. UMAP is used when available; otherwise the notebook falls back to t-SNE.

8 Evaluation Measures

The notebook implements the three required retrieval metrics:

$$\text{Recall@K} = \frac{\#\text{relevant retrieved documents}}{\#\text{relevant documents}}$$

$$\text{Precision@K} = \frac{\#\text{relevant retrieved documents}}{\#\text{retrieved documents}}$$

$$\text{MRR@K} = \frac{1}{\text{rank of the first relevant document}}$$

For Phase 1 comparison, the notebook also reports runtime and average latency per query. For Phase

2, it augments the retrieval summary with classifier accuracy, and computes a combined score as

$$\text{CombinedScore} = \frac{1}{4}\left(\text{Recall} + \text{Precision} + \text{MRR} + \text{Accuracy}\right).$$

9 Phase 1 Experimental Protocol

Only `queries_train.json` has relevance labels, while `queries_test.json` is the unlabeled competition test set. For that reason, the corrected notebook does not create a second query-side "test split" inside the labeled data. Instead, the labeled training queries are split into:

- 228 split-train queries ($\approx 70\%$);
- 99 validation queries ($\approx 30\%$).

This correction matters for two reasons. First, it keeps the experimental story aligned with the provided files: labeled training queries for development, unlabeled Kaggle test queries for final export. Second, with only 327 labeled queries available, removing an extra internal query test split leaves more supervised data for the classifier.

Phase 1 comparison is run on the **validation split**. For each model and for several values of K, the notebook stores:

- top-k document ids;
- top-k retrieval scores;
- Recall@K, Precision@K, and MRR@K;
- runtime and latency.

The explicit arrays requested by the assignment are preserved in the notebook as:

- `topk_indices_tfidf`, `topk_scores_tfidf`;
- `topk_indices_bm25`, `topk_scores_bm25`;
- `topk_indices_embedding`, `topk_scores_embedding`.

9.1 Phase 1 validation results

The best validation result for each Phase 1 model is shown in Table 6. In the current notebook, all three models reached their best combined Phase 1 score at $K = 1000$.

Model	K	Recall@K	Precision@K	MRR@K	Latency/query
TF-IDF	1000	0.53142	0.00462	0.21852	429.84 ms
BM25+	1000	0.58701	0.00513	0.29833	414.46 ms
Embeddings	1000	**0.85902**	**0.00827**	**0.50574**	**6.41 ms**

Table 6: Best validation result per Phase 1 model.

The Phase 1 comparison leads to three main conclusions.

- **Embeddings were the strongest retrieval model.** They gave the highest Recall@K and MRR@K, which is consistent with the vocabulary-mismatch nature of the task.
- **BM25+ clearly improved over TF-IDF on lexical retrieval.** This is expected because BM25+ handles document-length variation and repeated term saturation more explicitly.

- **The embedding implementation was also the fastest in this notebook.** Because document embeddings were cached and query scoring was chunked, dense retrieval was much faster than the current TF-IDF and BM25 loops in local execution.

System stage	Recall@K	Precision@K	MRR@K	Interpretation
TF-IDF baseline	0.53142	0.00462	0.21852	A reasonable lexical starting point, but still weak on short-query vocabulary mismatch.
BM25+ lexical model	0.58701	0.00513	0.29833	The best lexical baseline, showing that better term weighting helps but does not close the gap.
Embedding retrieval	**0.85902**	**0.00827**	**0.50574**	The first major jump in performance; semantic matching becomes the key direction for the project.

Table 7: How the retrieval project progressed during Phase 1.

10 Phase 2: Category Classification

10.1 Classifier model

The final notebook uses a **TF-IDF + LinearSVC** classifier. This choice is motivated by the fact that Phase 2 is a *classification* problem rather than a retrieval problem: given one text, the system must assign one category label.

TF-IDF converts each text into a sparse weighted vocabulary vector. Terms that are frequent inside one text but not frequent across the whole corpus receive larger weights. This representation is a strong fit for category prediction in this dataset because categories are often signaled by distinctive technical vocabulary. In other words, exact words such as package names, tools, commands, platforms, or domain-specific expressions often carry strong evidence about the correct label.

LinearSVC is a linear Support Vector Machine classifier. In practice, it learns a set of weights over the TF-IDF features and gives each class a score, then predicts the class with the highest score. Linear SVMs are a standard choice for text classification because they work very well with high-dimensional sparse vectors, train quickly, and provide a strong multiclass baseline that is easy to reproduce.

It is also useful to explain why the other project models were not used as the primary classifier. **BM25+** is fundamentally a query-document matching function, not a supervised multiclass classifier. It is well suited for ranking documents against a query, but using it for category prediction would require a more indirect design such as matching a text against category prototypes. **Dense embeddings** are excellent for Phase 1 retrieval because they reduce vocabulary mismatch and capture semantic similarity, but an embedding-based classifier would add extra modeling complexity and tuning effort here.

For this reason, the final design keeps the strongest semantic model where it matters most for retrieval, while using a lighter and more direct supervised model for category prediction. Since the classifier is used as a *soft reranking signal* rather than as the main retrieval engine, **TF-IDF + LinearSVC** offers the clearest tradeoff between accuracy, speed, interpretability, and methodological simplicity.

10.2 Training data choice

The assignment explicitly asks whether both documents and queries can be used, and whether the classifier works equally well for both. To answer those questions directly, the notebook trains the classifier on:

- a training split of the document corpus;
- the split-train portion of the labeled training queries.

On the query side, the corrected protocol evaluates on validation only, because `queries_test.json` is the actual unlabeled competition test set and should not be re-described as a second offline query test split. The classifier therefore evaluates separately on:

- held-out documents;
- validation queries.

The same trained classifier is then applied to `queries_test.json` only to generate submission-time category hints, not offline accuracy scores. This setup still makes it possible to compare performance on long documents and short queries rather than discussing only one of them.

For the query side, the notebook now evaluates two representations on the same split:

- a **text-only** variant based on `title + text`, used as the default Phase 2 setting because it stays closest to the wording of the brief;
- a **text + tags** variant, reported separately as an ablation because the raw JSON files do expose query tags.

This change is important methodologically. If the tag-aware classifier performs better, the report can mention that result explicitly without pretending it was the only reasonable interpretation of test-time information.

10.3 Classification metrics

The notebook reports:

- overall accuracy;
- a classification report with precision, recall, and F1-score per category.

Accuracy is computed as:

$$\text{Accuracy} = \frac{\#\text{correct predictions}}{\#\text{all predictions}}.$$

The final measured accuracies are shown in Table 8.

Source	Variant	Split	Accuracy
Queries	text-only (`title + text`)	validation	0.91919
Queries	text + tags	validation	**1.00000**
Documents	text-only classifier pipeline	holdout	0.98604

Table 8: Measured classification accuracies from the final notebook.

These numbers show two useful facts. First, document classification is indeed easier than query classification because documents contain more lexical evidence. Second, query tags add strong category information. However, we kept the **text-only** query classifier as the active Phase 2 setting because it stays closer to the assignment wording and to the retrieval-side query representation. The text-plus-tags result is therefore reported as a strong ablation, not silently substituted for the main pipeline.

11 Using the Classifier to Improve Retrieval

The professor asks how the classifier can be integrated into the retrieval system. The final notebook uses a simple and transparent strategy:

1. predict the category of each query;
2. retrieve top-k candidates with TF-IDF, BM25+, or embeddings;

3. normalize retrieval scores within each query;
4. add a soft bonus to documents whose category matches the predicted query category;
5. rerank the candidates.

This approach is easy to justify. If the category prediction is correct, the bonus increases the likelihood that same-category documents move upward in the ranked list. If the classifier is wrong, the bonus can hurt ranking quality, which is exactly why the Phase 2 comparison must be measured rather than assumed.

In the final notebook, the main reranking path uses the **text-only** query classifier by default. The tag-aware version remains available as a reported comparison, not as an unqualified replacement for the brief-aligned setting.

11.1 Phase 2 validation results

Table 9 reports the best Phase 2 validation result for each retrieval model after applying the category bonus.

Model	K	Recall@K	Precision@K	MRR@K	Accuracy	Combined	Latency/query
TF-IDF + classifier	1000	0.53142	0.00462	0.23132	0.91919	0.42164	424.15 ms
BM25+ + classifier	1000	0.58701	0.00513	0.29618	0.91919	0.45188	364.14 ms
Embeddings + classifier	1000	**0.85902**	**0.00827**	**0.50720**	0.91919	**0.57342**	**10.37 ms**

Table 9: Best validation result per Phase 2 model with classifier-aware reranking.

The main reason the embedding model remains best in Phase 2 is that the category bonus does not replace retrieval quality; it only nudges the ordering. Strong candidate generation is still the dominant factor, so the best Phase 1 retriever also remains the best Phase 2 retriever.

It is also important to interpret the Phase 2 gains correctly. At $K = 1000$, Recall and Precision remain almost unchanged for embeddings because the same candidate pool is preserved. The main retrieval-side gain is a small improvement in MRR, while the larger increase in the combined score comes from adding the classifier accuracy term required by the Phase 2 evaluation formula.

Roadmap step	Active system	Evidence	Why it mattered
Phase 1 baseline	TF-IDF retrieval	MRR@1000 = 0.21852	Established a simple reference point for the whole project.
Phase 1 improved baseline	BM25+ retrieval	MRR@1000 = 0.29833	Confirmed that stronger lexical weighting helps but is still not enough.
Phase 1 breakthrough	Embedding retrieval	MRR@1000 = 0.50574	Showed that semantic retrieval is the central improvement for this dataset.
Phase 2 final validation system	Embeddings + text-only classifier + soft bonus	Combined = 0.57342	Became the selected model because it is strong, simple, and easy to justify.
Corrected notebook public submission	Regenerated `solutions_SeaFour.csv` submission	Kaggle = **0.60007**	Confirms that the corrected protocol remains competitive on the public leaderboard.

Table 10: Project evolution from the first baseline to the final public leaderboard result.

12 Validation-Selected Submission and Kaggle Export

The best Phase 2 validation configuration is:

- retriever: **embeddings**;
- classifier variant: **text-only**;
- validation-selected evaluation depth: $\mathbf{K = 1000}$;
- submission export depth: **7500**.

Because the corrected protocol reserves `queries_test.json` as the only query-side test set, the notebook does not report an additional query holdout score after model selection. Instead, the selected validation winner is exported directly to the unlabeled Kaggle queries. The document classifier still keeps its own document-holdout evaluation reported earlier.

Split / Score	Recall@K	Precision@K	MRR@K	Accuracy	Combined / Public Score
Validation-selected system ($K = 1000$)	0.85902	0.00827	0.50720	0.91919	0.57342
Latest Kaggle public leaderboard (corrected notebook submission)	—	—	—	—	**0.60007**

Table 11: Current validation-selected system and the latest confirmed public Kaggle score.

Two design choices deserve explanation.

- **Why keep the text-only classifier?** Because it is the most defensible brief-aligned setting: the retrieval query and the classifier both rely on the actual query wording rather than on extra metadata injected at test time.
- **Why remove the extra query-side holdout split?** Because the dataset already provides labeled development queries through `queries_train.json` + `qgts_train.json` and a separate unlabeled competition test file through `queries_test.json`. Calling an extra internal slice of the labeled queries "test" blurs that protocol and wastes scarce supervised examples.
- **Why export 7500 documents for submission after selecting $K = 1000$ on validation?** Because $K = 1000$ was the best depth in the offline evaluation grid, while the final CSV export still benefits from a deeper ranked list for the competition output format. The notebook therefore separates `EVALUATION_TOP_KS` from `SUBMISSION_TOP_K`.

This protocol is cleaner than fitting and evaluating repeatedly on the same labeled queries, and it matches the logic expected in a reproducible retrieval pipeline:

1. develop and compare on training-derived splits;
2. choose the best validation configuration;
3. keep `queries_test.json` untouched until export;
4. export the final Kaggle file.

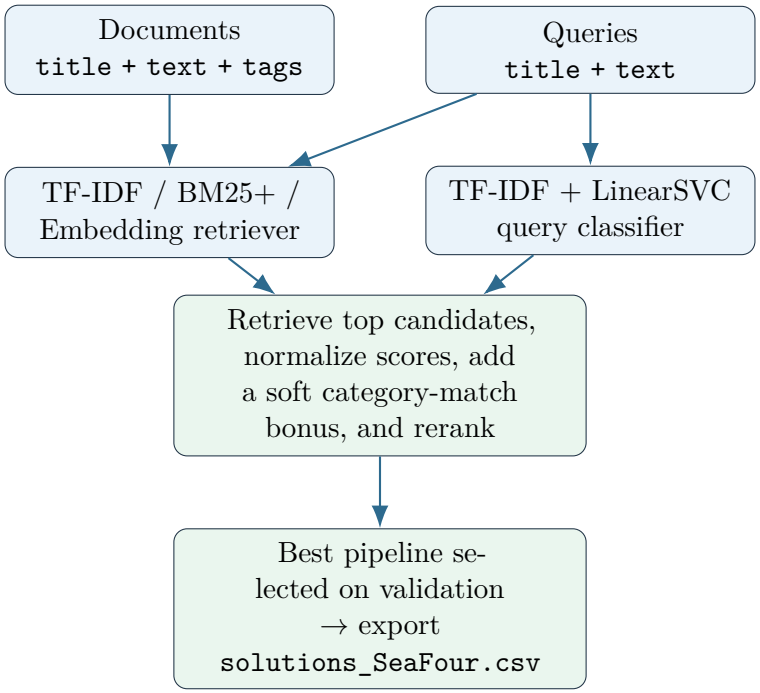

Figure 3: Final pipeline used to produce the submitted CSV.

13 Reproducibility

The final notebook includes several reproducibility choices:

- cached document embeddings;
- cached TF-IDF and BM25 artifacts;
- explicit runtime path detection for local, Colab, and Kaggle environments;
- explicit split fractions and random seeds;
- schema-aware qrels parsing that matches the competition files;
- stable category-preserving sampling for visualization and downsampling;
- notebook-level documentation through section markdown and structured outputs.

The cross-encoder extension remains disabled in the final notebook because the active submission path is intentionally centered on the assignment-visible pipeline: TF-IDF, BM25+, embeddings, classification, classifier-aware reranking, and submission export.

14 AI Usage Declaration

Large language model assistance was used during the preparation of this revised notebook/report package. In particular, LLM tools were used to:

- help reorganize and clean parts of the notebook and project documentation;
- suggest debugging fixes and consistency checks for the local/Kaggle runtime setup;
- help revise and polish sections of the written report.

The final methodological choices, interpretations, and submission artifacts remain the responsibility of the team. No LLM was used to access hidden Kaggle labels or any privileged evaluation data.

15 Conclusion

The final notebook and this report align with the full brief:

- they present the dataset and preprocessing explicitly;
- they compare TF-IDF, BM25+, and embeddings directly;
- they evaluate embeddings structurally and visually;
- they add category classification with document/query comparison;
- they use the classifier to improve retrieval in a way that is measurable and easy to justify;
- they preserve a reproducible path to Kaggle submission.

Empirically, the strongest active pipeline in the final notebook is the embedding retriever combined with the brief-aligned text-only classifier and a soft category bonus. Under the corrected protocol, this configuration achieved a best Phase 2 validation combined score of **0.57342**, and the regenerated Kaggle submission obtained a confirmed public score of **0.60007**. These results still support the main conclusion of the project: semantic retrieval is essential for this corpus, and a lightweight category-aware reranking step can improve the final system without making the pipeline unnecessarily complex.